Parts of an Expectation

When you see an expectation like the following one:

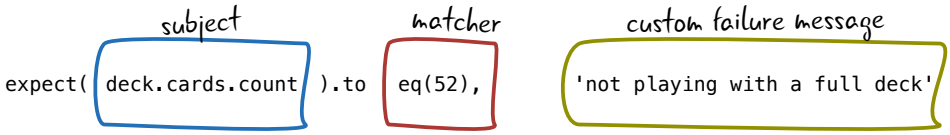
10-exploring-rspec-expectations/02/parts_of_an_expectation.rb

```
expect(deck.cards.count).to eq(52), 'not playing with a full deck'
```

…you'll notice several forms of punctuation in use: parentheses, dots, and whitespace. While there's some variety here, the syntax consistently uses just a few simple parts:

- A *subject*—the thing you're testing—that is, an instance of a Ruby class
- A *matcher*—an object that specifies what you expect to be true about the subject, and provides the pass/fail logic
- (optionally) A custom failure message

These parts are held together with a bit of glue code: `expect`, together with either the `to` or `not_to` method.



Let's get a feel for how these parts work by trying them out in an IRB session:

```
$ irb
```

To use rspec-expectations, you have to `require` the library, and then `include` the `RSpec::Matchers` module. Normally, rspec-core does this for you, but when you use rspec-expectations in another context, you need to bring it in yourself:

```
>> require 'rspec/expectations'
=> true
>> include RSpec::Matchers
=> Object
```

With that done, you can now create an expectation:

```
>> expect(1).to eq(1)
=> true
>> expect(1).to eq(2)
RSpec::Expectations::ExpectationNotMetError:
expected: 2
     got: 1

(compared using ==)

    « backtrace truncated »
```